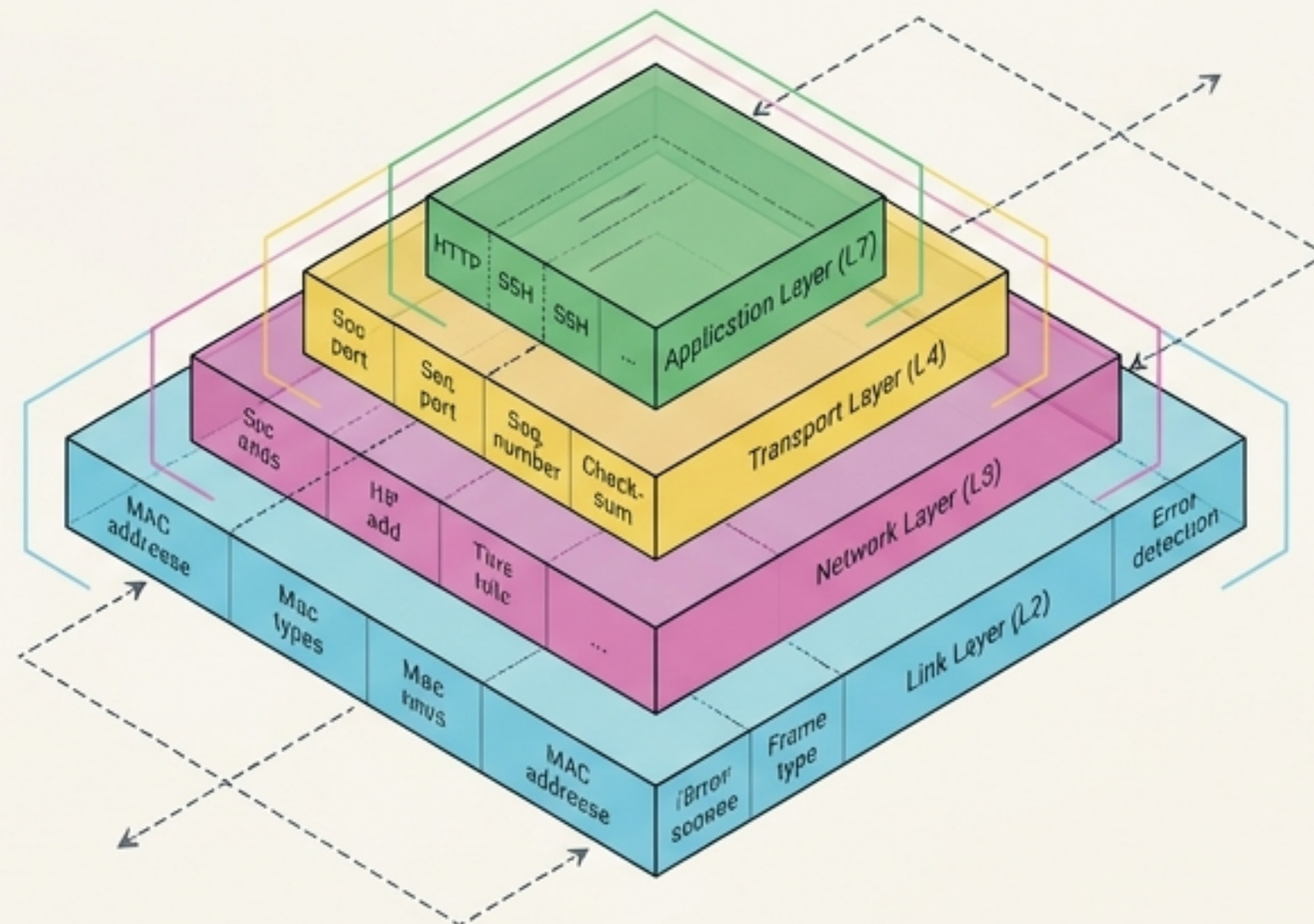


ANATOMY OF A PACKET

From Bare Metal to Application Data

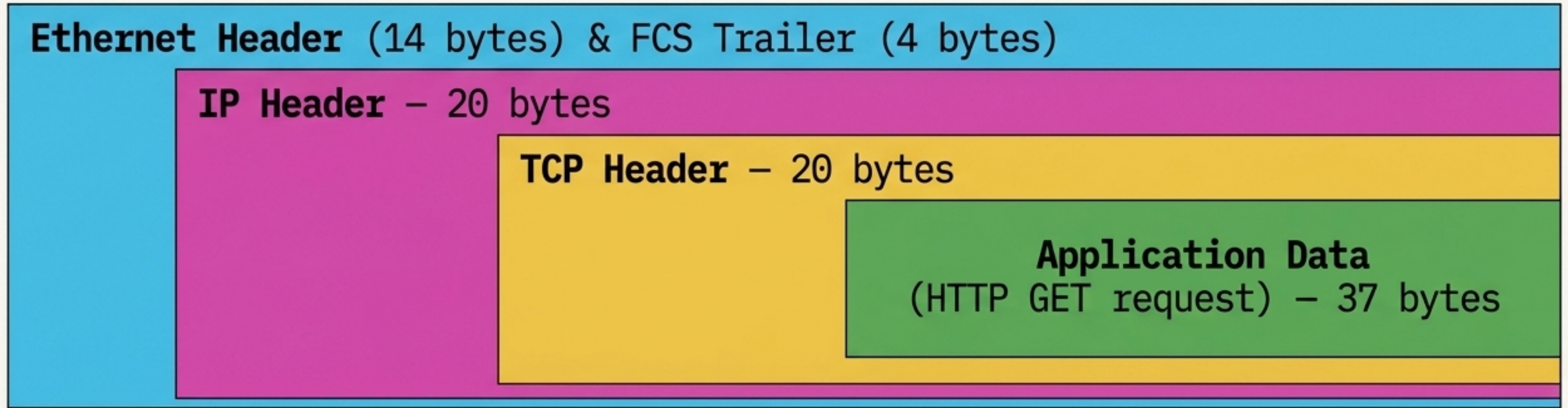
A digital forensics dive into encapsulation, layer independence, and the hidden mechanics of network transit.



Layer	Protocol	PDU	Key Address
Layer 7/6/5 (TCP/IP: App OSI: App/Pres/Sess)	HTTP	Data	URL
Layer 4 (TCP/IP: Transport OSI: Transport)	TCP/UDP	Segment	Ports (e.g., 80)
Layer 3 (TCP/IP: Network OSI: Network)	IPv4/IPv6	Packet	IP (e.g., 192.168.1.10)
Layer 2 (TCP/IP: Link OSI: Data Link)	Ethernet/ARP	Frame	MAC (e.g., aa:bb:cc...)
Layer 1 (TCP/IP: Physical OSI: Physical)	802.11 / Cables	Bits	(Signals)

Every tool in network forensics relies on understanding exactly which PDU and address type belongs to which layer.

The Encapsulation Tax



Total on wire: ~95 bytes

**Encapsulation overhead for this
37-byte payload is ~60%**

For small requests,
the envelope is
heavier than the
letter.

Forensic X-Ray: Hex to Header Mapping

```
00: FF FF FF FF FF FF AA BB CC DD EE FF 08 00 45 00
10: 00 28 00 01 00 00 40 06 7C CD C0 A8 01 0A 5D B8
20: D8 22 CB 20 00 50 00 00 00 00 00 00 00 00 00 50 02
```

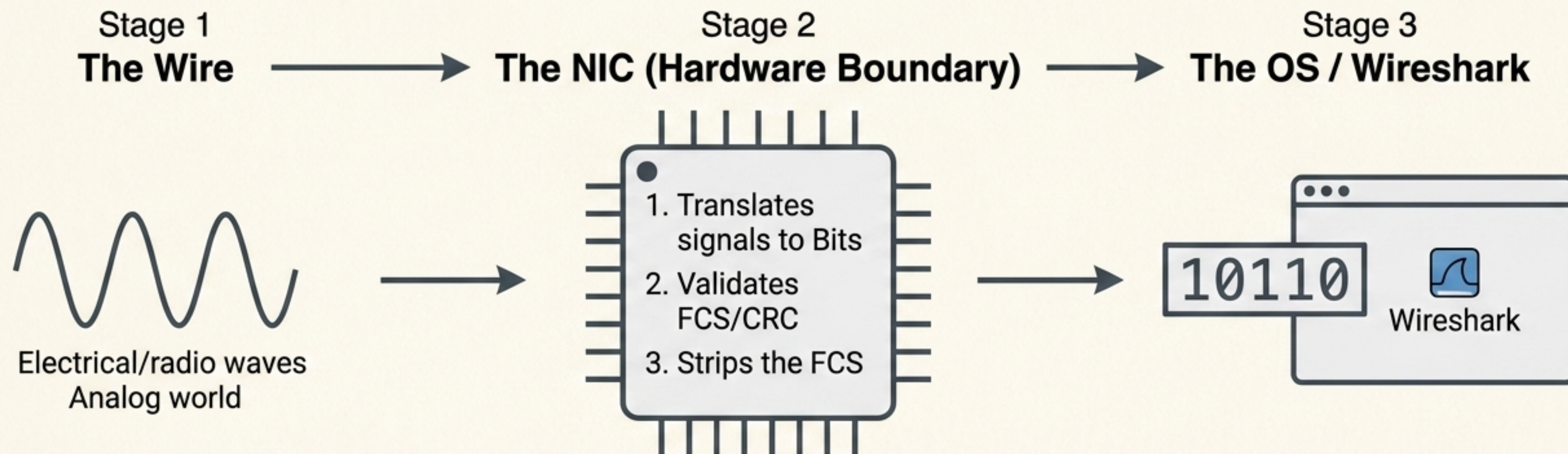
Destination MAC (Broadcast)

Source MAC

EtherType (IPv4 indicator)

Source IP (192.168.1.10)

The Ghost Physical Layer



By the time Wireshark captures a packet, the Physical layer is already gone. The NIC has converted the signal to bits, validated the checksum, and destroyed the Layer 1 evidence. We only observe the aftermath.

Hardware vs. Software Error Handling

Layer 2: Ethernet FCS/CRC

Mechanism:

Frame Check Sequence (Math: CRC32)

Domain:

Hardware (NIC)

Action on Error:

Silent drop. No notification, no logs, invisible to the OS.

Philosophy:

"Detect and destroy."

Layer 4: TCP Sequence/ACK

Mechanism:

Sequence and Acknowledgment numbers

Domain:

Software (OS TCP/IP Stack)

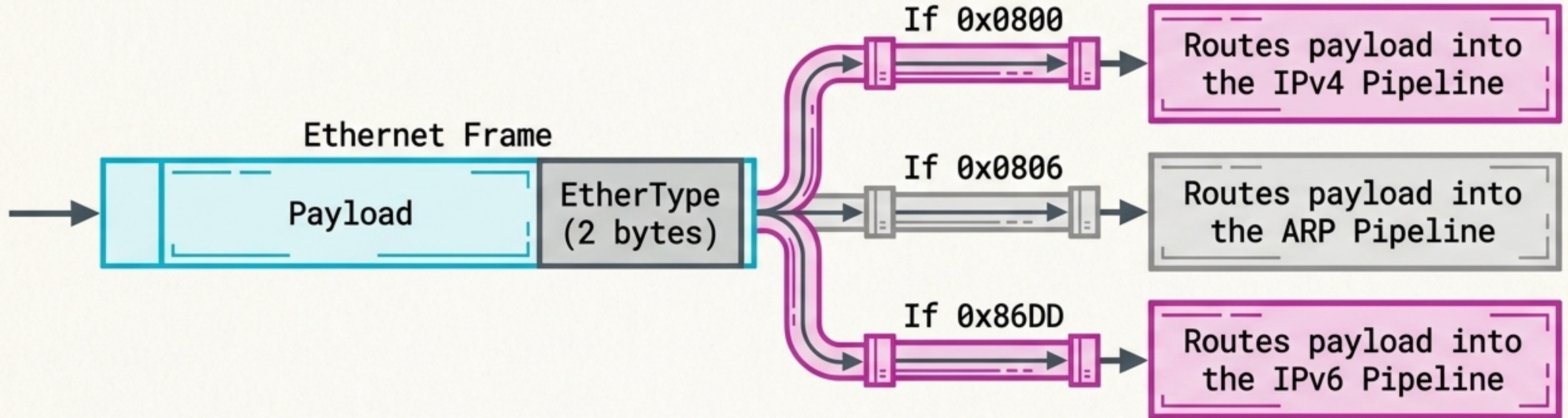
Action on Error:

The sender detects a missing ACK and automatically retransmits the lost segment. Visible in Wireshark.

Philosophy:

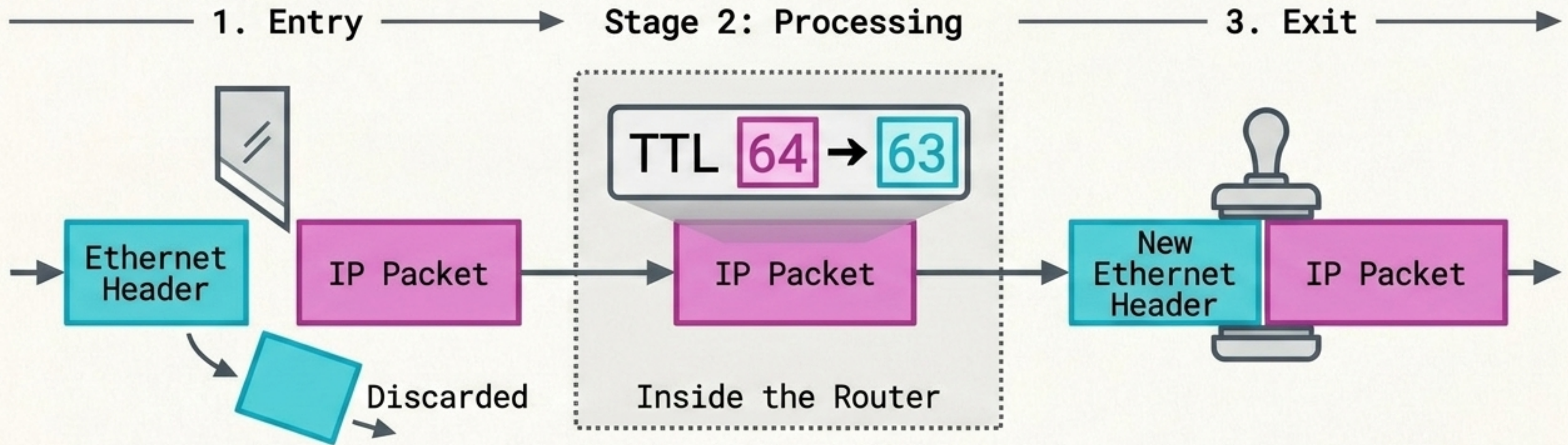
"Detect and recover."

The Layer Handoff (EtherType)



Ethernet has no idea what an IP address is. It simply carries a payload, reads a 2-byte hex code, and passes it to the correct handler. Layer modularity is a mechanical reality, not just a concept.

The Router Guillotine



Ethernet frames are mortal; they only survive a single hop. IP packets are immortal (mostly). At every hop, the router destroys the old frame and builds a completely new one.

Forensic Proof: Tracing the Hops

The Math

Starting Server TTL: 64

Arriving Packet TTL (Captured via TCPDump): 54

$$64 - 54 = 10 \text{ Hops.}$$

The CLI Proof

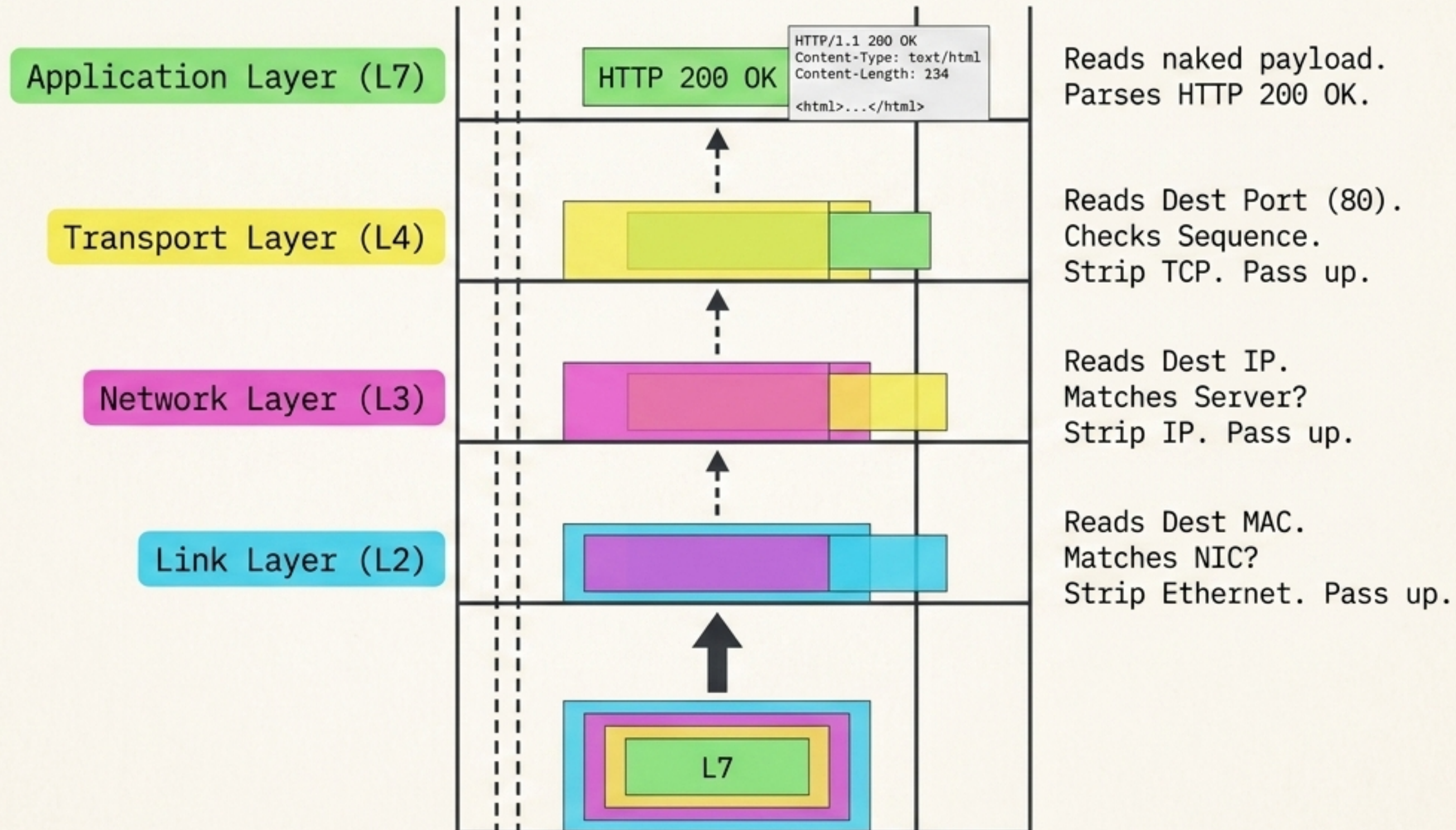
```
user@forensics:~$ traceroute example.com
```

1	192.168.1.1	0.412 ms	6	203.0.113.101	22.198 ms
2	10.0.0.1	1.205 ms	7	198.51.100.5	31.655 ms
3	172.16.5.1	4.891 ms	8	203.0.113.75	42.021 ms
4	203.0.113.1	9.332 ms	9	198.51.100.12	50.443 ms
5	198.51.100.25	15.760 ms	10	93.184.216.34	58.865 ms

Because the router decrements the IP TTL by 1 before building a new Ethernet frame, we can calculate exactly how many times our packet was stripped and rebuilt across the internet.

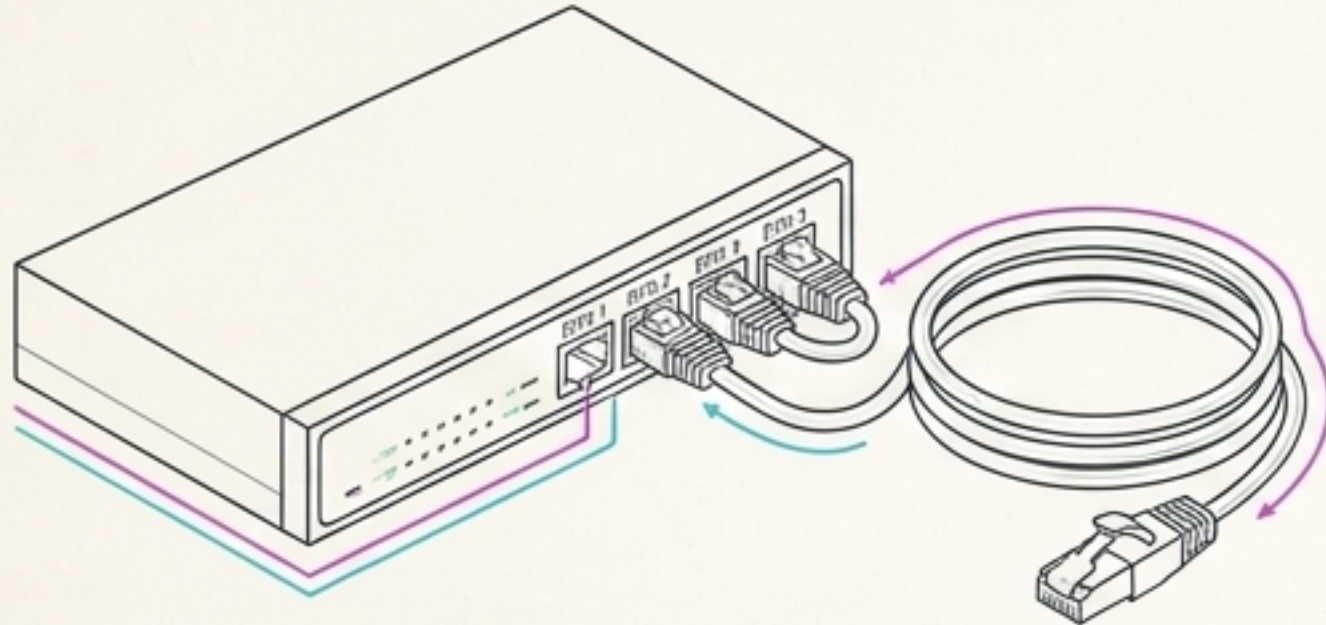
The Decapsulation Elevator

Peeling the headers at the destination.



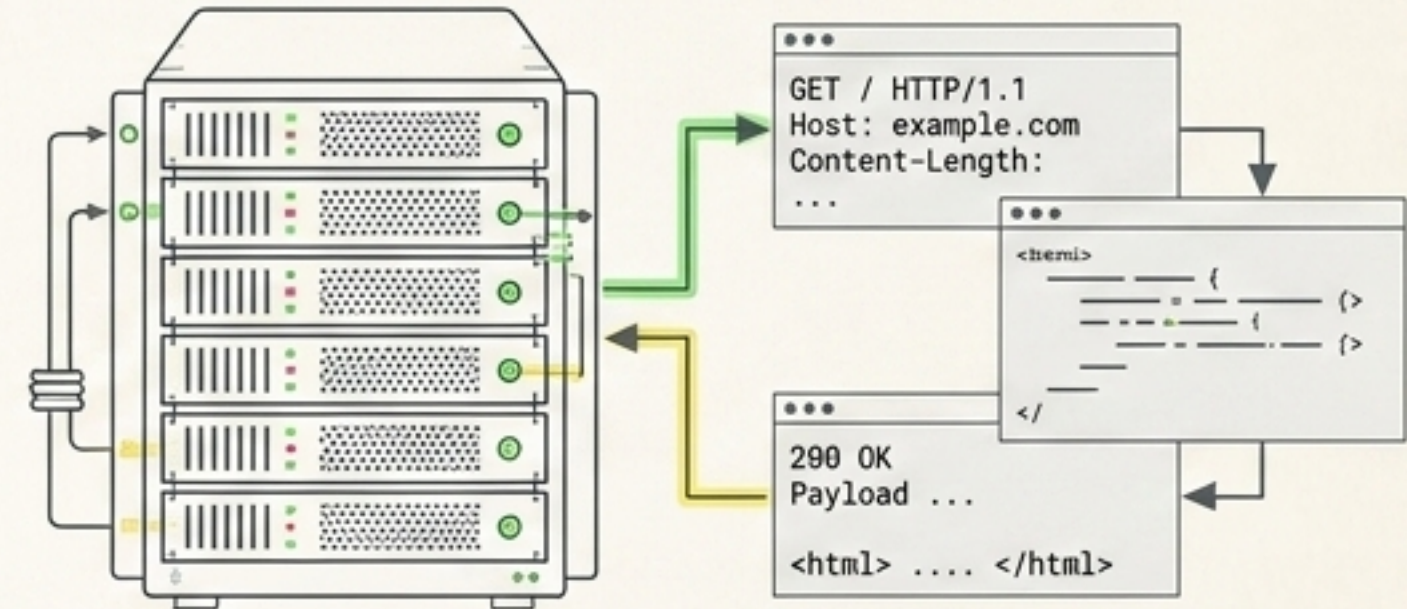
The Blueprints of the Web

Hardware Blueprint



- **Entity:** IEEE (Institute of Electrical and Electronics Engineers)
- **Standard:** 802.3 (Ethernet)
- **Role:** Dictates how Cisco switches and Juniper routers exchange electrical frames without negotiation.

Software Blueprint

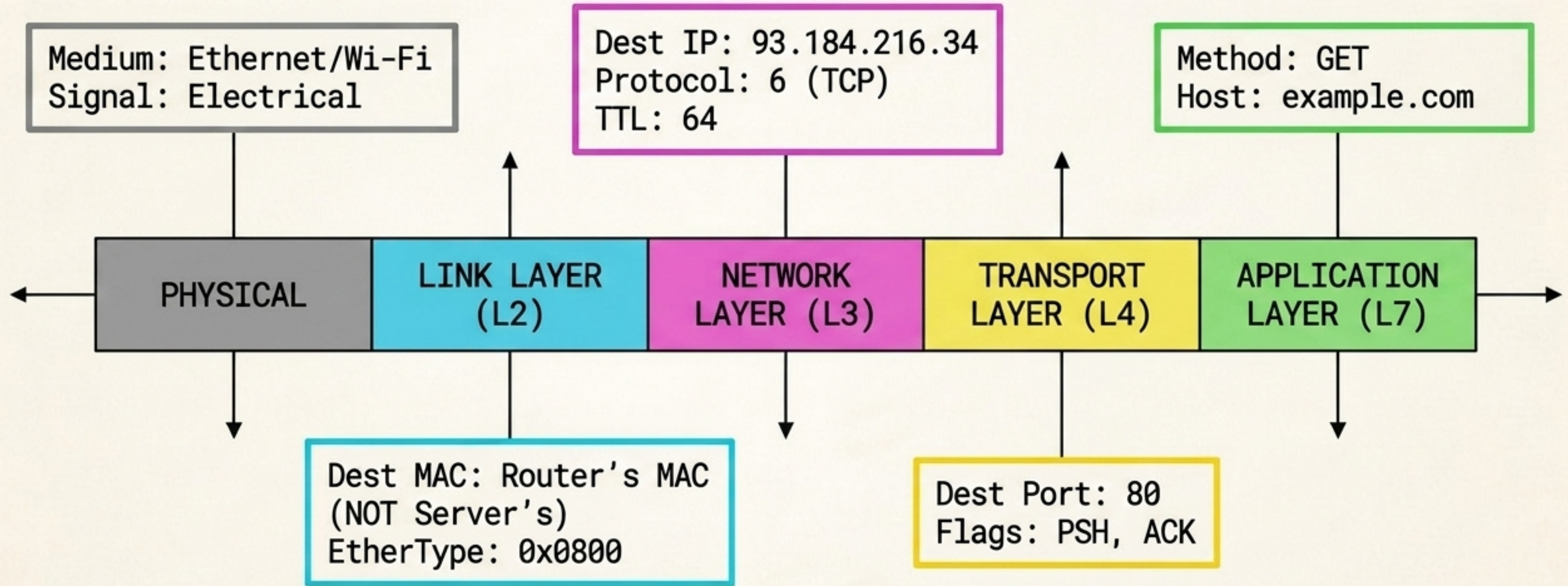


- **Entity:** IETF (Internet Engineering Task Force)
- **Standard:** RFC 2616 (HTTP/1.1)
- **Role:** Defines every field, behavior, and error condition so a Python script can talk to an Nginx server.

The internet works because competitors agree on the rules. Manufacturers implement IEEE for the metal; developers implement IETF RFCs for the logic.

End-to-End Packet Dossier

A complete X-Ray of a single HTTP request on the wire.



The Forensic Toolkit (tshark Reference)

Ethernet

- e eth.src (Source MAC)
- e eth.dst (Dest MAC)
- e eth.type (EtherType)

IP

- e ip.src (Source IP)
- e ip.dst (Dest IP)
- e ip.ttl (Time to Live)

TCP

- e tcp.srcport (Source Port)
- e tcp.dstport (Dest Port)
- e tcp.flags (TCP Flags)

App / Filters

- e http.request.method (HTTP Method)
- Y "tcp.flags.syn == 1" (Filter SYN only)

Understanding the stack is theory. Extracting the stack from the command line is cybersecurity.